12/12/23

Ejercicio 1 (2 puntos)

¿Cuál es la principal ventaja y la principal desventaja de las tablas de páginas invertidas? Justifique.

La ventaja principal es que ocupa menos espacio en memoria, ya que hay una entrada por página física en vez de por cada página virtual. Por ejemplo, con un procesador de 32b, una memoría física de 1GB y páginas de 4KB tendríamos 2^18 entradas con una TPI y 2^20 entrada con una tabla de páginas normal. La principal desventaja es que se debe recorrer toda la tabla en cada acceso, aumentando el costo de traducción.

Ejercicio 2 (4 puntos)

El ITBA adquirió una embarcación autopilotada y desea sacar a pasear a sus alumnos por las turquesas aguas de Puerto Madero. Para ello se solicita proveer código tanto para la embarcación como para los alumnos para sincronizar el embarque.

Los alumnos llegan al puerto y esperan a la embarcación. Cuando llega, todos embarcan (ejecutan `embarcar()`), pero cualquier alumno que llegue durante el embarque deberá esperar la próxima vuelta. La capacidad de la embarcación es de 20 personas, si hay más de 20 personas esperando, algunos tendrán que esperar la próxima vuelta. Una vez que todos los alumnos embarcaron, la embarcación parte (ejecuta `partir()`). Si no hay alumnos esperando cuando llega la embarcación, esta parte inmediatamente.

- `embarcar()` y `partir()` no se pueden modificar ni se puede asumir que modifican variables creadas por ustedes.
- No es necesario que los alumnos ejecuten `embarcar()` concurrentemente (de hecho, la embarcación tiene solo 1 escotilla).
- No es necesario modelar el desembarque, asuma que una vez que parte, la embarcación vuelve vacía al puerto.
- No es necesario modelar la creación de procesos / threads, un entry point, API de threads, liberación de recursos, etc. Enfoquémonos en lo importante.
- No puede haber calls entre alumnos y embarcación. Son procesos separados.
- La solución deberá estar libre de deadlock, condiciones de carrera, inanición y busy wait.

El código (sin sincronizar) que ejecuta cada alumno es el siguiente:

```
while 1
  embarcar()
```

El código (sin sincronizar) que ejecuta la embarcación es el siguiente:

```
while 1
  partir()
```

Complete el código con las primitivas de sincronización y/o variables que considere necesarias para resolver las restricciones establecidas.

Para una devolución más clara se recomienda agregar números de línea a la solución, incluso espacios en blanco.

```
sem_t mutex_fila = 1;
sem_t embarque = 1;
sem_t puede_embarcar = 0;
sem_t ya_embarco = 0;
global int fila = 0; // no dice que no vale memoria compartida
```

```
void alumno() {
  while(1) {
      wait(embarque);
      post(embarque);

      wait(mutex_fila);
      fila++;
      post(mutex_fila);

      wait(puede_embarcar);
      embarcar();
      post(ya_embarco);
    }
}
```

```
void barco() {
  while(1) {
      wait(embarque);

      wait(mutex_fila);
      int aux = 20;
      while(fila-- && aux--) {
        post(puede_embarcar);
        wait(ya_embarco);
      }
      post(mutex_fila);
      post(embarque);

      partir();
    }
}
```

Ejercicio 3 (2 puntos)

Diseñe un sistema de archivos basado en i-nodes tal que el tamaño máximo de archivo soportado sea aproximadamente 1 gigibyte. Se adjuntan las potencias de 2.

10	1.024	13	8.192	16	65.536	19	524.288	22	4.194.304	25	33.554.432	28	268.435.456
11	2.048	14	16.384	17	131.072	20	1.048.576	23	8.388.608	26	67.108.864	29	536.870.912
12	4.096	15	32.768	18	262.144	21	2.097.152	24	16.777.216	27	134.217.728	30	1.073.741.824

Direcciones de 4B y bloques de 4KB

Entonces en una bloque hay 2^12/4 = 2^10 direcciones

=> direccionamiento indirecto simple brinda 2^10 x 4KB = 4MB

=> direccionamiento indirecto doble brinda 2^10 x 2^10 x 4KB = 4GB -> me paso

No me da redondo, me queda mejor si tengo que cada bloque tiene 2^9 direcciones => 2^9 x 2^9 x 4KB = 1GB

=> Direcciones de 8B y bloques de 4KB => hay 2^9 punteros por bloque => Una entrada indirecta doble aporta 1GB

=> 10 entradas simples (estandar y no aporta tamaño significativo) y 1 entrada indirecta doble

8/7/24

EJERCICIO 1) Dar y explicar 4 ventajas de usar memoria virtual en vez de no usar ninguna abstracción de memoria.

1- Permite a los procesos utilizar más memoria de la que hay realmente.

2- Permite ejecutar múltiples programas que no están completamente en momeria, ya que se va cargando lo que cada uno necesita.

3- Permite que distintos procesos puedan compartir páginas, como es el caso con las librerías.

4- Brinda un mecanismo de seguridad entre procesos ya que cada uno solo puede acceder a las direcciones que tiene mapeado en su EDV. Un proceso no puede leer lo que cargó otro en memoria.

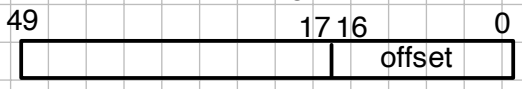
5- Evita condiciones de carrera cuando 2 procesos utilizan la misma dirección, ya que está mapeada a ≠ direcciones físicas.

EJERCICIO 3)
3.1 - Se tienen direcciones virtuales de 2 ^ 47 bytes y paginas de 16KB. Crear un sistema de paginación multinivel en el que el tamaño de las tablas de paginas sean un multiplo del tamaño de las paginas. Describir brevemente como se utiliza una dirección virtual con este sistema.
3.2 - Que beneficios nos da utilizar tablas de paginas

30 bits para dividir (47b × dirección)

3 niveles de 10b ⇒ $2^{14}/2^{10} = 2^4 = 16B$

2^47 / 2^14 = 2^33 páginas $\quad 16KB = 2^{4+10+3} = 2^{17}b \Rightarrow 17\ bits\ de\ offset$



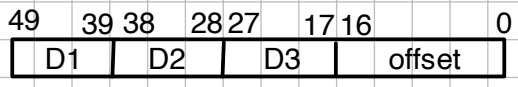
=> debo dividir 33 bits en directorios múltiplos de 16KB

Ahora debo definir un tamaño de entrada X adecuado, tomo que una entrada es de 8B.

=> 2^14 / 2^3 = 2^11 entradas en una página.

=> cantidad de niveles es 33b / 11b = 3 niveles de sistema de paginación

=>



Una dirección de 50b se divide en los primeros 11b a la entrada en D1, los segundos 11b a la entrada en D2 y los terceros 11b a la entrada en D3. Por último, los 17b menos significativos son el offset dentro de la página.

3.2) Utilizar tablas de páginas multinivel es útil cuando tenemos un EDV muy grande y ocuparía mucha memoria tenerlo cargado completo. Cargando únicamente las tablas de páginas que se están utilizando ocupa menos memoria.

EJERCICIO 4) 4.1 - Describir brevemente que hace que un proceso sea CPU-Bound
4.2 - Describir brevemente como distintos valores de quantum pueden afectar a un sistema interactivo
4.3 - Dado un tiempo t fijo, explicar porque la función del working set w(k,t) es no decreciente

4.1) Un proceso CPU-Bound es uno que tiene bursts de uso de CPU largos, es decir que pasa su mayor parte del tiempo utilizando el procesador.

4.2) Con un quantum muy largo, el sistema deja de ser responsivo y con uno muy corto, se realizarán muchos context switches disminuyendo notablemente el throughput.

4.3) El working set es el conjunto de páginas referenciadas en las últimas k referencias en un tiempo t. Es una función no decreciente pues el proceso cada vez va referenciando nuevas páginas, decrementa la velocidad en la que va requiriendo nuevas páginas, pero el conjunto no disminuye su tamaño.

Semáforos (2.5 puntos)

La siguiente es una implementación simplificada de semáforos que utiliza la instrucción en assembler ***xchg*** dentro de la función ***_xchg***, la cual intercambia los valores de ***sem_value*** y ***new_value*** atómicamente y retorna el valor de ***sem_value*** previo al intercambio.

```
extern int _xchg(int *sem_value, int new_value);

void sem_wait(sem *s){
  while(_xchg(&(s->value), s->value - 1) <= 0)
    _xchg(&(s->value), s->value + 1);
}

void sem_post(sem *s){
  _xchg(&(s->value), s->value + 1);
}
```

Determine si esta solución es correcta. En caso de considerarla incorrecta de un ejemplo que ilustre el problema y sugiera una solución. Para esto puede crear funciones y variables auxiliares, pero deberá utilizar la función ***_xchg*** provista. La espera activa se asume correcta.

El problema es que en el wait se calcula s->value -1 antes de hacer el xchg,

esto podria generar una race condition en la cual, hay 2 procesos/hilos T1, T2

que hacen wait con s->value = 0.

se hace s->value - 1 = -1 en T1 y todavia no se ejecuta el xchg

se hace s->value - 1 = -1 en T2 y se hace el xchg, ahora sem->value = -1

T1 hace el xchg y el valor queda -1

en ambos el xchg devuelve <= 0 por lo que hacen la suma, cada uno suma 1

ahora sem->value = 1

el proximo en correr, le va a dar > 0 el xchg de la condicion del while

por lo que saldria del wait sin que nadie haya hecho post

```c
typedef struct {
    volatile int value;
    volatile int lock;   // 0 = libre, 1 = tomado
} sem;

static inline void spin_lock(sem *s) {
    while (_xchg((int*)&s->lock, 1) == 1) {
        /* busy wait */
    }
}

static inline void spin_unlock(sem *s) {
    _xchg((int*)&s->lock, 0);
}
```

```c
void sem_wait(sem *s) {
    for (;;) {
        spin_lock(s);
        if (s->value > 0) {
            s->value--;
            spin_unlock(s);
            return;
        }
        spin_unlock(s);
        /* busy wait (podrías meter pause/yield si existiera) */
    }
}

void sem_post(sem *s) {
    spin_lock(s);
    s->value++;
    spin_unlock(s);
}
```

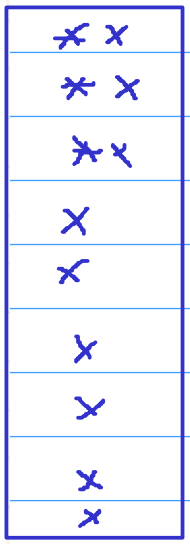
TLB (2 puntos)

Considere que el siguiente programa se ejecuta en una computadora con un tamaño de página de 2^{12} bytes y un TLB de 2^6 entradas.

```
1. int arr[ARR_SIZE];
2. int i = 0;
3. while(i < ARR_SIZE){
4.   arr[i] = arr[i] + 1;
5.   i += I_INCREMENT;
6. }
```

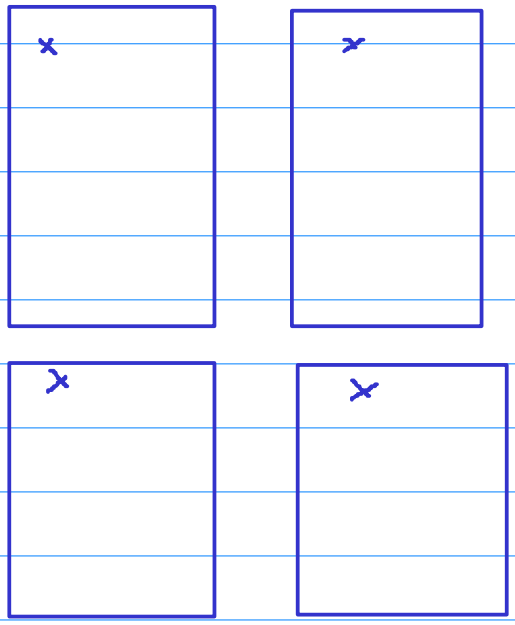
Indique qué valores de ARR_SIZE y I_INCREMENT causarían un *miss* en el TLB por cada iteración del ciclo. Justifique.

Si este ciclo se repitiera múltiples veces, ¿su respuesta seguiría siendo la misma? Justifique



paginas -> 2^12 B

TLB -> 2^6 entradas



quiero i=0 -> miss, i=1 -> miss, ...

lo que podemos hacer es que el array sea grande como 4 paginas y que el incremento sea tal que cada acceso te da en una pagina diferente

$$ARR_SIZE = \frac{\cancel{4} \cdot 2^{12}}{sizeof(int)} = 2^{12}$$

$$INCREMENT = \frac{pagesize}{sizeof(int)} = \frac{2^{12}}{2^2} = 2^{10}$$

o #entradas + 1

Si se ejecuta varias veces, como tiene 2^6 > 4 entradas, ya no haria mas misses en las proximas iteraciones, para que sigan habiendo misses el array deberia ocupar 2*#entradas paginas, osea el ARR_SIZE deberia ser 2*#entradas*pagesize/sizeof(int) = 2^15 para que tenga que borrar todas las entradas de la TLB 2 veces, suponiendo que el algoritmo de remplazo es LRU.